INFRASTRUCTURE AS CODE · COMPARATIF 2026

Terraform Enterprise vs Terraform vs OpenTofu

Licences, moteur, plateforme, coûts, migration — puis l'écosystème associé : Checkov, KICS, Trivy, TFLint, OPA/Conftest, Sentinel, Infracost, Terragrunt, TACOS…

État au 5 novembre 2026 · Terraform 1.16 · OpenTofu 1.12 · Terraform Enterprise 2.x · HCP Terraform

Utopios · Mohamed Aijjou

v1.0

SOMMAIRE

Déroulé

Partie 1 — Comparatif

1. Ce qu'on compare vraiment (moteur vs plateforme)
2. Chronologie : BSL, fork, rachat IBM
3. Licences & gouvernance
4. Comparatif global
5. Moteur : Terraform vs OpenTofu, fonctionnalité par fonctionnalité
6. Exclusivités OpenTofu / exclusivités Terraform
7. Plateforme : Terraform Enterprise & HCP Terraform
8. Coûts
9. Migration & matrice de décision

Partie 2 — Écosystème

10. La chaîne d'outillage IaC
11. Scanners de sécurité : Checkov, KICS, Trivy (tfsec), Terrascan
12. Policy as Code : OPA/Conftest vs Sentinel
13. Qualité : TFLint, terraform-docs, pre-commit
14. Tests : `terraform test` / `tofu test`, Terratest
15. Coûts : Infracost
16. Orchestration : Terragrunt, Terramate, Atmos
17. TACOS : Atlantis, Spacelift, env0, Scalr, Terrakube…
18. Pipeline GitLab CI complet & synthèse

01

Le comparatif

Distinguer le **moteur** (la CLI qui calcule le plan et applique) de la **plateforme** (qui orchestre, gouverne et audite les runs).

PÉRIMÈTRE

Ce qu'on compare vraiment : 4 produits, 2 catégories

Terraform Enterprise n'est pas « un autre Terraform » : c'est une plateforme qui exécute la CLI Terraform.

MOTEUR

Terraform CE

- CLI `terraform` (Community Edition)
- Licence **BSL 1.1** depuis 1.6
- Gratuit
- HashiCorp, groupe **IBM**

MOTEUR

OpenTofu

- CLI `tofu`, fork de Terraform 1.5.x
- Licence **MPL 2.0**
- Gratuit
- Linux Foundation, **CNCF Sandbox**

PLATEFORME SaaS

HCP Terraform

- Ex-« Terraform Cloud »
- Hébergé par HashiCorp
- Facturé au **RUM** (ressources gérées)
- Exécute Terraform CE

PLATEFORME self-hosted

Terraform Enterprise

- HCP Terraform installé chez vous
- Docker, Podman, Kubernetes, Nomad
- Air-gap possible
- Licence commerciale, sur devis

Conséquence pour le comparatif : TFE vs OpenTofu compare une plateforme à un moteur. L'équivalent d'une TFE côté OpenTofu, c'est **OpenTofu + une plateforme tierce** (TACOS : Spacelift, env0, Scalr, Terrakube, Atlantis…) ou **OpenTofu + votre CI** (GitLab CI + state GitLab). On compare donc sur deux axes : moteur (Terraform ↔ OpenTofu) et plateforme (TFE/HCP ↔ CI/TACOS).

CONTEXTE

Chronologie : de la licence BSL au paysage 2026

- **2014** — Terraform 0.1 (MPL 2.0)
- **Juin 2021** — Terraform 1.0 : promesses de compatibilité v1.x
- **10 août 2023** — HashiCorp passe ses produits en **BSL 1.1**
- **Sept. 2023** — Manifeste OpenTF → fork **OpenTofu** sous Linux Foundation
- **10 janv. 2024** — OpenTofu 1.6 GA
- **Avril 2024** — OpenTofu 1.7 : chiffrement du state ; réponse publique à une lettre de mise en demeure de HashiCorp
- **27 févr. 2025** — **IBM finalise le rachat de HashiCorp** (6,4 Md$)
- **2025** — OpenTofu entre en **CNCF Sandbox**
- **Juin 2025** — OpenTofu 1.10 : registries OCI, locking S3 natif
- **Nov. 2025** — Terraform 1.14 : list resources, `terraform query`, actions
- **Déc. 2025** — CDKTF déprécié ; OpenTofu 1.11 (ephemeral, `enabled`)
- **31 mars 2026** — Fin du plan **Free (legacy)** HCP Terraform → free tier 500 RUM
- **1er avr. 2026** — Fin de support de TFE en mode Replicated
- **Avr.–mai 2026** — Terraform 1.15 · **TFE 2.0** (Stacks, SCIM 2.0) · **OpenTofu 1.12**
- **26 août 2026** — Terraform 1.16 (actions on destroy, import au niveau module)
- **En cours** — OpenTofu 1.13 en développement (supportée jusqu'au 1er août 2027)

LICENCES

BSL 1.1 vs MPL 2.0 : qui est réellement concerné ?

Terraform ≥ 1.6 — Business Source License 1.1

- Code source **visible**, usage interne et en production **autorisé**
- **Interdit** : proposer un produit/service **concurrent** de HashiCorp qui héberge ou embarque Terraform (« Additional Use Grant »)
- **Change Date** : chaque version repasse en licence open source (MPL 2.0) 4 ans après sa publication
- Terraform ≤ 1.5.x reste sous MPL 2.0
- TFE / HCP Terraform : licences commerciales classiques

OpenTofu — Mozilla Public License 2.0

- Licence open source reconnue par l'OSI
- Copyleft **faible**, au niveau fichier : modifier un fichier MPL → le publier sous MPL
- Fork, redistribution et offre commerciale **libres**
- Gouvernance neutre : Linux Foundation / CNCF, Technical Steering Committee
- Les **providers** (AWS, Azure, Google…) restent majoritairement MPL 2.0 → utilisables par les deux moteurs

PROFIL	IMPACT JURIDIQUE DE LA BSL	IMPACT STRATÉGIQUE
Entreprise utilisatrice (IaC interne)	**Faible** — usage autorisé	Dépendance à un éditeur unique (IBM), feuille de route non ouverte
ESN / MSP qui opère l'IaC de ses clients	**À analyser** selon l'offre (hébergée ou non)	Risque contractuel → OpenTofu souvent retenu
Éditeur de plateforme IaC (TACOS)	**Fort** — cas visé par la clause	Raison d'être du fork OpenTofu

Ce tableau n'est pas un avis juridique : faire valider par le service juridique toute offre commerciale embarquant Terraform.

GOUVERNANCE & CYCLE DE VIE

Qui décide, à quel rythme, avec quel support ?

CRITÈRE	TERRAFORM CE	TERRAFORM ENTERPRISE	OPENTOFU
Propriétaire	HashiCorp, groupe IBM	HashiCorp, groupe IBM	Linux Foundation (CNCF Sandbox)
Décision	Éditeur ; issues publiques, feuille de route interne	Éditeur ; fonctionnalités livrées d'abord sur HCP Terraform	Technical Steering Committee, RFC publiques
Contributions	Acceptées sous CLA, priorités éditeur	Code fermé	Ouvertes, contributeurs multi-entreprises
Cadence mineure	~ tous les 4 mois (1.14 nov. 2025, 1.15 avr. 2026, 1.16 août 2026)	Versions 1.x puis 2.x, patchs réguliers	~ tous les 6 mois (1.10 juin 2025, 1.11 déc. 2025, 1.12 mai 2026)
Durée de support	Pas de LTS formel ; promesses de compatibilité v1.x	TFE 2.0 : support standard 2 ans, support étendu en option	Fenêtre publiée par série (1.12 → 1er févr. 2027, 1.13 → 1er août 2027)
Support commercial	Via un contrat HCP / TFE	Support IBM / HashiCorp inclus	Via des éditeurs tiers (Spacelift, env0, Scalr, Harness, Gruntwork…)
Binaires	`releases.hashicorp.com`, signés GPG	Images de conteneur (registre HashiCorp)	GitHub releases, paquets, `ghcr.io/opentofu`, signatures cosign + GPG

À retenir : le versionnage des deux CLI se ressemble (1.x), mais **les numéros ne correspondent plus** : Terraform 1.12 ≠ OpenTofu 1.12. Toujours comparer par fonctionnalité, jamais par numéro de version.

VUE D'ENSEMBLE

Comparatif global

DOMAINE	TERRAFORM CE	TERRAFORM ENTERPRISE	OPENTOFU
Nature	CLI	Plateforme (UI, API, runs distants) + CLI Terraform	CLI
Licence / coût	BSL 1.1 · gratuit	Commerciale · sur devis	MPL 2.0 · gratuit
Exécution	Poste, CI	Runs distants, file d'attente, agents (réseaux privés)	Poste, CI, TACOS
State	Backends : S3, azurerm, gcs, http (GitLab), pg, consul…	Géré, versionné, historisé, consultable dans l'UI	Mêmes backends + OCI, chiffrement natif
Chiffrement state	✗ côté client — repose sur le backend	Chiffré au repos par TFE (Vault interne)	✓ côté client (AES-GCM, KMS, PBKDF2, OpenBao)
Policy as Code	Externe (OPA/Conftest)	✓ Sentinel + OPA natifs, niveaux d'enforcement	Externe (OPA/Conftest) ou via TACOS
Registry privée	Registry publique / tiers	✓ Private registry, no-code modules, tests de modules	✓ Registries OCI (Harbor, GHCR, ECR…), network mirror
RBAC / SSO	✗ (droits = ceux de la CI)	✓ Teams, projects, SAML SSO, SCIM 2.0	✗ (droits = ceux de la CI / TACOS)
Audit	Logs CI	✓ Audit trail, rôle *site auditor*	Logs CI ; traces OpenTelemetry
Drift	`plan -detailed-exitcode` planifié	✓ Health assessments (drift + continuous validation)	Idem CE, ou via TACOS
Credentials cloud	Variables CI, OIDC de la CI	Dynamic provider credentials (OIDC workload identity)	Variables CI, OIDC de la CI
Air-gap	Provider mirror	✓ Installation air-gap supportée	Provider mirror, registry OCI interne
Moteur exécuté	Terraform	Terraform uniquement (versions sélectionnables par workspace)	OpenTofu

Terraform vs OpenTofu : fonctionnalités communes (version d'apparition)

Les deux moteurs partagent le langage HCL, le protocole de plugins des providers et le format du state (avant les fonctionnalités divergentes).

FONCTIONNALITÉ	TERRAFORM	OPENTOFU	COMMENTAIRE
Tests natifs `*.tftest.hcl`	1.6	1.6	`terraform test` / `tofu test`
Mocks de providers dans les tests	1.7	1.8	`mock_provider`, `override_resource`
Bloc `removed`, `import` avec `for_each`	1.7	1.7	Refactoring sans destruction
Provider-defined functions	1.8	1.7	`provider::aws::arn_parse(...)`
Ephemeral resources (secrets hors state)	1.10	1.11	Valeur jamais écrite dans le state ni le plan
Write-only attributes	1.11	1.11	Ex. mot de passe de base de données
Locking S3 natif (`use_lockfile`)	1.10 (exp.) / 1.11	1.10	Plus besoin de table DynamoDB
Variables dans `source` / `version` de module et dans le backend	1.15 (dynamic sources)	1.8 (early evaluation)	OpenTofu en avance de ~2 ans
Dépréciation de variables / outputs	1.15	1.10	Avertissement aux appelants du module
Fonction `convert()`	1.15	1.13 (à venir)	Conversion de type explicite
Windows ARM64 officiel	1.15	1.13 (à venir)	Fin des builds 32 bits annoncée des deux côtés

Fonctionnalités divergentes : là où les moteurs se séparent

EXCLUSIVITÉS OPENTOFU	
Chiffrement du state et du plan côté client	1.7
Fichiers `.tofu` (prioritaires sur `.tf`)	1.8
`for_each` sur les blocs `provider`	1.9
Option `-exclude` (inverse de `-target`)	1.9
Providers et modules depuis un registry **OCI**	1.10
Traces OpenTelemetry	1.10
Meta-argument `enabled` (fini `count = x ? 1 : 0`)	1.11
`prevent_destroy` piloté par variable	1.12
`lifecycle { destroy = false }` sur la ressource	1.12
Sorties humaine + JSON simultanées	1.12

EXCLUSIVITÉS TERRAFORM	
Stacks (multi-composants, multi-déploiements) + CLI `terraform stacks`	1.13 + HCP / TFE 2.0
List resources + `terraform query` (découverte & génération de config)	1.14
Bloc `action` (opérations hors CRUD)	1.14
Actions sur `before_destroy` / `after_destroy`	1.16
`import` au niveau module	1.16
`terraform graph -format=mermaid`	1.16
`terraform_data` avec bloc `store` (plan → apply)	1.16
Intégration native HCP/TFE (bloc `cloud`, Sentinel)	—
CDKTF	**déprécié déc. 2025**

Conséquence : dès qu'un projet utilise une exclusivité, il devient **non portable** vers l'autre moteur (ex. un state chiffré par OpenTofu est illisible par Terraform ; un bloc `action` est une erreur de syntaxe pour OpenTofu). Statut au 05/11/2026, versions stables uniquement.

Chiffrement du state et du plan (OpenTofu ≥ 1.7)

```
terraform {
  encryption {
    # Clé dérivée d'une passphrase (≥ 16 caractères)
    key_provider "pbkdf2" "main" {
      passphrase = var.state_passphrase
    }
    # Alternative production : aws_kms, gcp_kms, openbao
    method "aes_gcm" "main" {
      keys = key_provider.pbkdf2.main
    }
    state {
      method   = method.aes_gcm.main
      enforced = true   # refuse tout state en clair
    }
    plan {
      method   = method.aes_gcm.main
      enforced = true
    }
  }
}

variable "state_passphrase" {
  type      = string
  sensitive = true
}
```

Ce qu'on montre

- `tofu apply` puis ouverture du `terraform.tfstate` : contenu **opaque** (JSON chiffré)
- `tofu show` fonctionne avec la passphrase (`TF_VAR_state_passphrase`)
- Sans la passphrase : erreur de déchiffrement

Pourquoi c'est utile

- Le state contient des secrets en clair (mots de passe, clés)
- Protection même si le bucket ou le backend HTTP est compromis

Limites

- Perte de la clé = **perte du state**
- Terraform ne peut plus lire ce state (retour arrière : `method "unencrypted"` + `fallback`)
- Les outils qui lisent le state brut ne le comprennent plus
- Rotation de clé : bloc `fallback`

Provider `for_each`, `enabled` et `prevent_destroy` dynamique

```hcl
# OpenTofu ≥ 1.9 : un provider par région
variable "regions" {
  type    = set(string)
  default = ["eu-west-1", "eu-west-3"]
}

provider "aws" {
  alias    = "by_region"
  for_each = var.regions
  region   = each.key
}

resource "aws_s3_bucket" "logs" {
  for_each = var.regions
  provider = aws.by_region[each.key]
  bucket   = "utopios-logs-${each.key}"
}
```

Test : ajouter une région à la variable → un provider et un bucket de plus, sans dupliquer de bloc. **Limite :** sans équivalent dans Terraform, le code n'est plus portable.

```hcl
# OpenTofu ≥ 1.11 : activation conditionnelle
variable "with_bastion" {
  type    = bool
  default = false
}

resource "aws_instance" "bastion" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    enabled = var.with_bastion
  }
}

# Référence : aws_instance.bastion.id
# (pas d'index [0] comme avec count)
```

Test : `-var with_bastion=true` → création ; repasser à `false` → destruction. Lisibilité supérieure à `count`.

```hcl
# OpenTofu ≥ 1.12 : protection par environnement
variable "protect" {
  type    = bool
  default = true
}

resource "aws_db_instance" "main" {
  identifier        = "app-db"
  engine            = "postgres"
  instance_class    = "db.t4g.micro"
  allocated_storage = 20

  lifecycle {
    prevent_destroy = var.protect
  }
}
# dev.tfvars  : protect = false
# prod.tfvars : protect = true
```

Test : `tofu destroy -var-file=prod.tfvars` → refusé ; avec `dev.tfvars` → accepté. Terraform exige une valeur littérale ici.

EXCLUSIVITÉS TERRAFORM · DÉMO

Actions, `terraform query` et Stacks

```hcl
# Terraform ≥ 1.14 : action hors CRUD
action "aws_lambda_invoke" "warmup" {
  config {
    function_name = aws_lambda_function.api.function_name
    payload       = jsonencode({ warm = true })
  }
}

resource "terraform_data" "release" {
  input = aws_lambda_function.api.version

  lifecycle {
    action_trigger {
      events  = [after_create, after_update]
      actions = [action.aws_lambda_invoke.warmup]
    }
  }
}
# Ou à la demande :
# terraform apply -invoke=action.aws_lambda_invoke.warmup
```

```hcl
# Terraform ≥ 1.14
# fichier discover.tfquery.hcl
list "aws_instance" "existing" {
  provider = aws
}

# Lister l'existant
$ terraform query

# Générer la configuration
# et les blocs import
$ terraform query \
    -generate-config-out=imported.tf

# Remplace en grande partie
# Terraformer pour l'import
# d'infrastructure existante
```

Stacks (HCP Terraform / TFE 2.0)

- **Components** : modules assemblés en un tout
- **Deployments** : même stack déployée N fois (régions, environnements, comptes)
- Gestion des dépendances et des valeurs inconnues (*deferred changes*)
- Orchestration des déploiements, réglée côté plateforme
- CLI : `terraform stacks` (1.13+)

Équivalents côté OpenTofu : Terragrunt (`terragrunt.stack.hcl`), Terramate, Atmos, ou les stacks d'un TACOS (Spacelift).

Limite : ces fonctionnalités exigent des providers récents qui implémentent les nouvelles RPC (actions, list resources). Leur couverture varie selon le provider et la ressource.

REGISTRIES & PROVIDERS

Providers, modules et registries : compatibilité réelle

Ce qui est partagé

- Même **protocole de plugins** (v5/v6) : un provider fonctionne avec les deux moteurs
- Les providers HashiCorp et partenaires restent sous MPL 2.0
- Les modules HCL sont compatibles tant qu'ils n'utilisent pas d'exclusivité
- `required_providers { aws = { source = "hashicorp/aws" } }` reste valable

Ce qui diffère

- Registry par défaut : `registry.terraform.io` vs `registry.opentofu.org`
- `.terraform.lock.hcl` : hachages à regénérer lors du changement de moteur (`tofu init -upgrade`)
- OpenTofu : providers et modules depuis un registry **OCI** (Harbor, GitLab, GHCR, ECR)
- TFE : **private registry** intégrée, avec modules *no-code*

BESOIN	TERRAFORM CE	TFE	OPENTOFU
Modules privés	Sources Git (`git::https://…?ref=v1.2.0`)	Private registry versionnée, tests de modules	Git, ou registry OCI (`oci://…`)
Environnement déconnecté	`provider_installation { network_mirror }`	Installation air-gap + registry interne	Network mirror ou registry OCI interne
Documentation pour assistants IA	Terraform MCP server (HashiCorp)	Terraform MCP server	OpenTofu Registry MCP server

PLATEFORME · TERRAFORM ENTERPRISE

Ce que TFE ajoute au-dessus de la CLI

Organisation

- Organizations, projects, workspaces
- Variable sets partagés
- Migration de workspaces entre organisations (2.0)

Workflow

- VCS-driven (GitLab, GitHub, Bitbucket, Azure DevOps)
- CLI-driven (bloc `cloud`)
- API-driven
- Speculative plans sur MR/PR

Gouvernance

- **Sentinel** et **OPA**, policy sets
- Advisory / soft / hard mandatory
- **Run tasks** : Snyk, Prisma Cloud, Infracost, Wiz…

Sécurité

- SAML SSO, **SCIM 2.0** (Okta, Entra ID)
- Teams & permissions fines
- Expiration obligatoire des tokens API (2.0)
- Dynamic provider credentials (OIDC)

Exploitation

- Agents (réseaux privés, on-prem)
- Health assessments : drift + continuous validation
- Ephemeral workspaces (auto-destroy)

Libre-service

- Private registry
- No-code modules
- Explorer : inventaire des workspaces, versions, modules

Audit

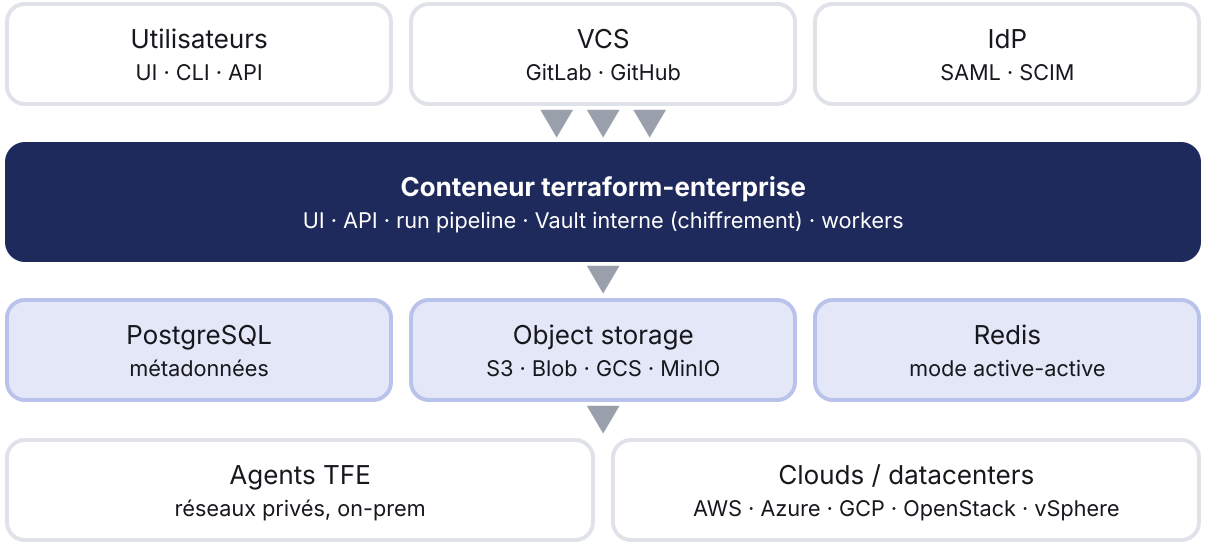
- Audit trail
- Rôle *site auditor* en lecture seule, sans accès aux données sensibles du state (2.0)
- Notifications au niveau projet (2.0)

Multi-env

- **Stacks** (2.0)
- Run triggers entre workspaces
- Diagnostics de santé et contrôles avant mise à niveau (2.0)

Ce que TFE ne fait pas : exécuter OpenTofu. Les fonctionnalités arrivent en général d'abord sur HCP Terraform, puis dans une version ultérieure de TFE.

Architecture et déploiement de TFE



ÉLÉMENT	OPTIONS
Runtimes	Docker, Podman, Kubernetes (EKS, AKS, GKE, OpenShift), Nomad
Modes	**Disk** (mono-nœud), **External services**, **Active-active** (Redis, multi-nœuds)
Installation	En ligne ou **air-gap** (image + licence)
Replicated	**Fin de support le 1er avril 2026** → migrer vers les Flexible Deployment Options
Versions	Mensuelles `v2025xx` → 1.x (2025) → **2.0** (mai 2026), support standard 2 ans
Mises à jour	Respecter les *required releases* en air-gap

> **Coût caché :** TFE est une application à exploiter — sauvegarde PostgreSQL et object storage, montées de version, supervision, certificats TLS, renouvellement de la licence.

PLATEFORME

HCP Terraform (SaaS) vs Terraform Enterprise (self-hosted)

CRITÈRE	HCP TERRAFORM	TERRAFORM ENTERPRISE
Hébergement	SaaS HashiCorp (régions US et UE)	Votre infrastructure (cloud ou on-prem)
Exploitation	Aucune	À votre charge (HA, sauvegardes, upgrades)
Nouvelles fonctionnalités	En premier	Avec décalage, par version
Localisation des states	Chez HashiCorp	Chez vous — **souveraineté**
Air-gap / réseau isolé	✗ (agents sortants possibles)	✓
Tarification	Free 500 RUM · Essentials · Standard · Premium (au RUM)	Contrat entreprise, sur devis
Cas typiques	Startups, équipes cloud-first sans contrainte de localisation	Défense, banque, santé, secteur public, industrie

Exemple : un industriel de la défense en réseau isolé n'a pas le choix entre les deux : seule TFE (ou une alternative self-hosted : Terrakube, Atlantis, GitLab CI) est envisageable.

COÛTS

Combien ça coûte vraiment ?

OFFRE	PRIX PUBLIC (OCT. 2026)
Terraform CE / OpenTofu	0 € de licence
HCP Terraform Free	500 ressources gérées, utilisateurs illimités, 1 run concurrent
HCP Essentials	~0,10 $ / ressource / mois
HCP Standard	~0,47 $ / ressource / mois
HCP Premium	~0,99 $ / ressource / mois
Terraform Enterprise	Sur devis (contrat annuel)

RUM (Resources Under Management) : ressources `managed` dans les states ; les data sources, `null_resource` et `terraform_data` ne sont pas comptées. Le plan Free legacy (illimité en ressources) a pris fin le 31 mars 2026.

RESSOURCES GÉRÉES	STANDARD (CALCUL)	PREMIUM (CALCUL)
1 000	~470 $/mois	~990 $/mois
5 000	~2 350 $/mois	~4 950 $/mois
10 000	~4 700 $/mois	~9 900 $/mois

Calcul théorique sur prix publics, hors remises et hors free tier.

Le gratuit n'est pas gratuit : avec CE ou OpenTofu, vous payez en temps d'ingénierie (pipelines, gestion des states, policy, RBAC via la CI, audit) et éventuellement un TACOS. Le bon calcul est le **coût total** : licence + exploitation + temps d'équipe.

Une seule ressource EKS « complète » (VPC, IAM, add-ons) peut représenter plusieurs centaines de RUM.

MIGRATION

Migrer de Terraform vers OpenTofu (et retour)

```
# 0. Prérequis : outils installés, accès au backend,
#    gel des déploiements pendant l'opération
$ terraform version            # noter la version source
$ tofu version

# 1. Sauvegarde du state
$ terraform state pull > backup-$(date +%F).tfstate

# 2. Point de départ propre
$ terraform plan               # attendu : No changes

# 3. Bascule
$ rm -rf .terraform
$ tofu init -upgrade           # regénère le lock file
$ tofu plan                    # attendu : No changes

# 4. Réécriture du state par OpenTofu
$ tofu apply

# 5. Mettre à jour la CI : image, commandes
#    terraform → tofu, cache .terraform
```

SITUATION DE DÉPART	DIFFICULTÉ
Terraform ≤ 1.5	**Simple** : migration directe
Terraform 1.6 → 1.9	**Simple** : suivre le guide de migration de la version cible
Terraform ≥ 1.10 avec ephemeral / write-only	**Moyenne** : OpenTofu ≥ 1.11 requis
Bloc `cloud`, Stacks, actions, `query`	**Refactoring** : remplacer par backend standard / Terragrunt
TFE / HCP Terraform	**Projet** : export des states (API), variables, Sentinel → OPA, runs → CI/TACOS

Retour arrière OpenTofu → Terraform : possible tant qu'aucune exclusivité OpenTofu n'est utilisée. Avec le chiffrement : repasser d'abord en `method "unencrypted"` via `fallback`, appliquer, puis changer de moteur.

Ce qu'on teste en démo : un `plan` sans changement des deux côtés, et le `serial` du state incrémenté après `tofu apply`.

DÉCISION

Matrice de décision

CONTEXTE / EXIGENCE	CHOIX RECOMMANDÉ	POURQUOI
Équipe réduite, CI existante (GitLab), pas d'exigence d'audit centralisé	OpenTofu ou Terraform CE	State GitLab + pipeline suffisent ; zéro licence
Secteur régulé, réseau isolé, RBAC/SSO/audit exigés, support éditeur requis	TFE	Gouvernance clé en main, installation air-gap, contrat de support
Même besoin de gouvernance, sans équipe pour héberger, SaaS accepté	HCP Terraform	Aucune exploitation ; attention au coût au RUM
ESN / MSP / éditeur qui opère de l'IaC pour des tiers	OpenTofu	Pas de risque lié à la clause BSL
Besoin de chiffrer le state côté client, de providers multi-régions	OpenTofu	State encryption, provider `for_each`
Besoin de Stacks, actions, `terraform query`, Sentinel	Terraform + HCP/TFE	Exclusivités HashiCorp
Stratégie open source / souveraineté, gouvernance self-hosted	OpenTofu + TACOS self-hosted	Terrakube, Atlantis, Digger, Spacelift self-hosted
Parc existant sous TFE avec Sentinel, satisfaction élevée	Rester sur TFE	Le coût de migration dépasse souvent le gain

Règle pratique : le choix du moteur (Terraform vs OpenTofu) est réversible tant qu'on évite les exclusivités ; le choix de la plateforme (TFE vs CI/TACOS) structure l'organisation pour des années.

02

L'écosystème associé

Scanners de sécurité, policy as code, qualité, tests, coûts, orchestration et plateformes : ce qui entoure le moteur dans une chaîne IaC de production.

VUE D'ENSEMBLE

La chaîne d'outillage IaC, étape par étape

1. Écrire
Complétion, docs des providers, assistants IA
terraform-ls · OpenTofu LSP · MCP servers

2. Qualité
Format, syntaxe, bonnes pratiques, docs
fmt · validate · TFLint · terraform-docs · pre-commit

3. Sécurité statique
Mauvaises configurations dans le HCL
Checkov · KICS · Trivy

4. Tests
Comportement du module
terraform/tofu test · Terratest

5. Plan
Plan sauvegardé + export JSON
plan -out · show -json

6. Policy
Règles d'entreprise sur le plan
OPA/Conftest · Sentinel

7. Coût
Estimation et écart de coût
Infracost

8. Revue
Commentaires MR/PR, approbation
Atlantis · TACOS · GitLab MR

9. Apply
Application du plan approuvé
CI · TFE · TACOS

10. Run
Drift, conformité continue
plan planifié · health assessments

Transverse — orchestration : Terragrunt, Terramate, Atmos (multi-environnements, dépendances, DRY).

Transverse — versions : tenv (Terraform, OpenTofu, Terragrunt, Atmos), tfenv, tfswitch.

SÉCURITÉ STATIQUE

Scanners IaC : comparatif

OUTIL	ÉDITEUR	LANGAGE	RÈGLES INTÉGRÉES	FORMATS	RÈGLES CUSTOM	POINT FORT	STATUT 2026
Checkov	Palo Alto (Prisma Cloud)	Python	1 000+	~12 : TF, plan JSON, CFN, K8s, Helm, Dockerfile, ARM/Bicep, CI…	Python, YAML	Analyse **en graphe** (relations entre ressources), scan du plan	Actif
KICS	Checkmarx	Go	2 400+ queries	22+ : TF, CFN, Pulumi, Ansible, K8s, Docker, OpenAPI, Crossplane…	Rego	Couverture la plus large ; codes de sortie par sévérité	Actif
Trivy `config`	Aqua Security	Go	Bibliothèque héritée de tfsec	TF, CFN, K8s, Helm, Dockerfile, ARM	Rego	Un seul binaire : IaC + images + SBOM + secrets + licences	Actif
tfsec	Aqua Security	Go	—	Terraform	Rego, JSON	Rapide, spécialisé Terraform	Fusionné dans Trivy
Terrascan	Tenable	Go	500+	TF, CFN, K8s, Helm, Kustomize	Rego	Admission controller K8s	Archivé nov. 2025
Snyk IaC	Snyk	—	Catalogue Snyk	TF, CFN, K8s, ARM	Rego	Intégration à la plateforme Snyk	Freemium

Besoin IaC seul, profondeur : Checkov.

Formats IaC variés : KICS.

Un outil pour tout (images + IaC) : Trivy.

Tous analysent le HCL et/ou le plan JSON → utilisables avec Terraform et OpenTofu. Vérifier le support des fichiers `.tofu` et des syntaxes récentes (provider `for_each`, `enabled`) dans la version utilisée ; à défaut, scanner le **plan JSON**.

Checkov : utilisation, exceptions, règle personnalisée

```
# Installation
$ pip install checkov          # ou image bridgecrew/checkov

# Scan du code HCL
$ checkov -d . --framework terraform

# Scan du plan (valeurs résolues, modules inclus)
$ tofu plan -out=plan.cache
$ tofu show -json plan.cache > plan.json
$ checkov -f plan.json --framework terraform_plan

# Filtrer, rapports pour la CI
$ checkov -d . --check CKV_AWS_18,CKV_AWS_21 \
    --skip-check CKV_AWS_144 \
    -o cli -o junitxml \
    --output-file-path console,checkov.xml

# Règles maison
$ checkov -d . --external-checks-dir ./policies

# Exception justifiée, dans le code :
resource "aws_s3_bucket" "logs" {
  #checkov:skip=CKV_AWS_18:Bucket de logs d'accès lui-même
  bucket = "utopios-access-logs"
}
```

```
# policies/s3_owner_tag.yaml
metadata:
  id: "CKV2_UTOP_1"
  name: "Les buckets S3 doivent porter le tag owner"
  category: "GENERAL_SECURITY"
definition:
  cond_type: "attribute"
  resource_types:
    - "aws_s3_bucket"
  attribute: "tags.owner"
  operator: "exists"
```

- **Montrer :** un bucket sans chiffrement ni versioning → `FAILED` avec l'ID de règle et le lien vers la doc
- **Pourquoi le plan :** le HCL contient des variables non résolues ; le plan JSON contient les vraies valeurs
- **Graphe :** règles `CKV2_*` vérifiant des relations (ex. bucket ↔ `aws_s3_bucket_public_access_block`)
- **Limites :** faux positifs fréquents, dépendance Python, ne voit pas l'état réel du cloud

KICS (Keeping Infrastructure as Code Secure)

```
# Exécution conteneurisée (aucune installation)
$ docker run --rm -v "$PWD":/path checkmarx/kics:latest \
    scan -p /path -o /path/kics-results \
    --report-formats json,sarif,html

# Échouer seulement à partir de HIGH
$ kics scan -p ./infra --fail-on high

# Restreindre le périmètre
$ kics scan -p ./infra --type Terraform \
    --exclude-paths ./infra/legacy \
    --exclude-queries 568a4d22-3517-44a6-a7ad-6a7eed88722c

# Exception dans le code
resource "aws_security_group_rule" "https" {
  # kics-scan ignore-line
  cidr_blocks = ["0.0.0.0/0"]
  ...
}
```

CODE DE SORTIE	SIGNIFICATION
0	Aucun résultat
60	Au moins un CRITICAL
50	Au moins un HIGH
40	Au moins un MEDIUM
30	Au moins un LOW
20	INFO uniquement

- **Montrer :** le rapport HTML, regroupé par sévérité et par catégorie
- **Pourquoi KICS :** un même outil pour Terraform, Ansible, Kubernetes, Dockerfile, OpenAPI
- **Règles :** queries Rego (OPA), extensibles
- **Limites :** pas d'analyse de graphe inter-ressources comme Checkov ; identifiants de règles en UUID, peu lisibles

Trivy (successeur de tfsec) et sécurité de la chaîne d'approvisionnement

```
# Mauvaises configurations IaC
$ trivy config ./infra

# Bloquant en CI sur HIGH et CRITICAL
$ trivy config --severity HIGH,CRITICAL \
    --exit-code 1 ./infra

# Valeurs d'un environnement
$ trivy config --tf-vars prod.tfvars ./infra

# Formats de rapport
$ trivy config --format sarif -o trivy.sarif ./infra

# Exception dans le code (ID hérité de tfsec)
#trivy:ignore:AVD-AWS-0086
resource "aws_s3_bucket_public_access_block" "site" {
  ...
}

# Migration depuis tfsec :
# tfsec .  →  trivy config .
# les identifiants AVD-* restent valables
```

- **Intérêt :** un seul outil pour IaC, images de conteneurs, SBOM, secrets, licences
- **Limite :** moins de règles IaC que Checkov, pas d'analyse de graphe
- **Combinaison fréquente :** Trivy (images) + Checkov (IaC)

Mars 2026 — incident supply chain : l'infrastructure de publication de Trivy a été compromise (tags de GitHub Actions détournés, fausses releases, images malveillantes) ; la GitHub Action de KICS a été touchée par la même campagne.

Bonnes pratiques qui en découlent

- Épingler les Actions et composants CI par **SHA de commit**, jamais par tag
- Tirer les images par **digest** (`@sha256:…`)
- Vérifier les binaires (`cosign verify-blob` , sommes de contrôle)
- Miroir interne des outils (Harbor, registry GitLab)

POLICY AS CODE

OPA / Conftest vs Sentinel

```rego
# policy/s3.rego — OPA / Conftest (Terraform et OpenTofu)
package main

import rego.v1

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "aws_s3_bucket"
  rc.change.actions[_] in {"create", "update"}
  not rc.change.after.tags.owner
  msg := sprintf("%s : tag 'owner' obligatoire", [rc.address])
}

# Exécution sur le plan JSON
$ conftest test plan.json -p policy/
```

```
# s3-owner.sentinel — TFE / HCP Terraform
import "tfplan/v2" as tfplan

s3 = filter tfplan.resource_changes as _, rc {
  rc.type is "aws_s3_bucket" and
  rc.mode is "managed" and
  (rc.change.actions contains "create" or
   rc.change.actions contains "update")
}

main = rule {
  all s3 as _, b {
    b.change.after.tags contains "owner"
  }
}
# Enforcement : advisory | soft-mandatory | hard-mandatory
```

CRITÈRE	OPA / CONFTEST	SENTINEL
Licence	Apache 2.0, projet CNCF gradué	Propriétaire, inclus dans HCP/TFE
Portée	Universel : Terraform, OpenTofu, Kubernetes (Gatekeeper), API, CI	Écosystème HashiCorp
Intégration	CI, TACOS, TFE (policy sets OPA)	Native dans le run TFE, overrides tracés
Scanner vs policy	Un scanner (Checkov…) applique des règles **génériques** de sécurité ; la policy applique des règles **d'entreprise** (tags, régions, tailles, coûts).	

QUALITÉ

TFLint, terraform-docs et pre-commit

```
# .tflint.hcl
plugin "terraform" {
  enabled = true
  preset  = "recommended"
}

plugin "aws" {
  enabled = true
  version = "0.40.0"
  source  = "github.com/terraform-linters/tflint-ruleset-aws"
}

# Commandes
$ tflint --init
$ tflint --recursive \
    --format compact
```

TFLint détecte ce que `validate` ignore : type d'instance inexistant, variables non utilisées, conventions de nommage. Rulesets AWS, Azure, Google.

```
# Génération de la doc d'un module
# (variables, outputs, providers)
$ terraform-docs markdown table \
    --output-file README.md \
    --output-mode inject \
    ./modules/vpc

# Dans README.md :
<!-- BEGIN_TF_DOCS -->
(tableau généré)
<!-- END_TF_DOCS -->

# Format et syntaxe natifs
$ tofu fmt -check -recursive
$ tofu validate
```

terraform-docs maintient la documentation des modules synchronisée avec le code — indispensable pour une registry privée.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.96.1
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_tflint
      - id: terraform_docs
      - id: terraform_checkov
      - id: terraform_trivy

# Avec OpenTofu :
# args: [--hook-config=--tf-path=tofu]

$ pre-commit install
$ pre-commit run -a
```

pre-commit déplace les contrôles sur le poste du développeur : retour en secondes, avant même la MR.

TESTS

Tests natifs vs Terratest

```
# tests/bucket.tftest.hcl — terraform test / tofu test
mock_provider "aws" {}

variables {
  bucket_name = "utopios-demo"
}

run "nom_du_bucket" {
  command = plan

  assert {
    condition     = aws_s3_bucket.this.bucket == "utopios-demo"
    error_message = "Nom de bucket inattendu"
  }
}

run "nom_invalide_refuse" {
  command = plan
  variables { bucket_name = "UPPER_CASE" }
  expect_failures = [var.bucket_name]
}

$ tofu test        # ou terraform test
```

```
// test/bucket_test.go — Terratest (Go)
package test

import (
  "testing"
  "github.com/gruntwork-io/terratest/modules/terraform"
  "github.com/stretchr/testify/assert"
)

func TestBucket(t *testing.T) {
  opts := &terraform.Options{
    TerraformDir:    "../examples/basic",
    TerraformBinary: "tofu",
  }
  defer terraform.Destroy(t, opts)
  terraform.InitAndApply(t, opts)
  name := terraform.Output(t, opts, "bucket_name")
  assert.Contains(t, name, "utopios")
}

$ go test ./test -timeout 30m
```

CRITÈRE	TESTS NATIFS (`.TFTEST.HCL`)	TERRATEST
Langage	HCL, pas de nouvelle compétence	Go
Infra réelle	Optionnelle (`command = plan` + mocks)	Oui, déploie réellement puis vérifie (HTTP, SSH, API)
Usage	Tests unitaires de modules, validations	Tests d'intégration / bout en bout

FINOPS

Infracost : le coût dans la merge request

```
# Prérequis : clé API gratuite pour la CLI
$ infracost auth login
# en CI : variable INFRACOST_API_KEY

# Estimation depuis le code
$ infracost breakdown --path ./infra

# Estimation depuis le plan JSON (valeurs résolues)
$ infracost breakdown --path plan.json \
    --format json --out-file infracost.json

# Écart par rapport à la branche principale
$ infracost breakdown --path ./infra \
    --format json --out-file baseline.json   # sur main
$ infracost diff --path ./infra \
    --compare-to baseline.json               # sur la MR

# Commentaire automatique sur la MR GitLab
$ infracost comment gitlab --path infracost.json \
    --repo "$CI_PROJECT_PATH" \
    --merge-request "$CI_MERGE_REQUEST_IID" \
    --gitlab-server-url "$CI_SERVER_URL" \
    --gitlab-token "$GITLAB_TOKEN" \
    --behavior update
```

Ce qu'on montre

- Passer `t3.micro` → `m5.2xlarge` : la MR affiche **+ X $/mois**
- Ressources à coût d'usage (S3, Lambda) : estimées via `infracost-usage.yml`

Pourquoi

- Rendre le coût visible **avant** l'apply, pas sur la facture du mois suivant
- Combinable avec une policy : refuser une MR au-delà d'un seuil

Limites

- Prix publics catalogue, sans remises négociées
- Ressources non supportées signalées mais non chiffrées
- Tableaux de bord et guardrails avancés : offre payante Infracost Cloud
- Fonctionne avec Terraform et OpenTofu ; existe en **run task** TFE

ORCHESTRATION

Terragrunt, Terramate, Atmos

```
# live/root.hcl — configuration commune
remote_state {
  backend = "s3"
  generate = {
    path      = "backend.tf"
    if_exists = "overwrite_terragrunt"
  }
  config = {
    bucket       = "utopios-tfstate"
    key          = "${path_relative_to_include()}/terraform.tfstate"
    region       = "eu-west-3"
    encrypt      = true
    use_lockfile = true
  }
}

# live/prod/vpc/terragrunt.hcl
include "root" {
  path = find_in_parent_folders("root.hcl")
}
terraform {
  source = "../../../modules/vpc"
}
inputs = {
  cidr = "10.20.0.0/16"
}

$ cd live/prod && terragrunt run --all plan
```

OUTIL	APPROCHE	MOTEURS
Terragrunt	Wrapper HCL : backend DRY, `dependency` entre unités, `run --all`, stacks (`terragrunt.stack.hcl`)	OpenTofu et Terraform (`tofu` utilisé en priorité si présent)
Terramate	Stacks, génération de code, **change detection** Git : n'exécute que ce qui a changé	Terraform, OpenTofu
Atmos	Configuration YAML en couches (Cloud Posse), catalogues de composants	Terraform, OpenTofu

Quand l'adopter ? Dès que plusieurs environnements × régions × comptes multiplient les backends et les variables. **Limite :** une couche d'abstraction de plus à former et à déboguer.

Pour les utilisateurs OpenTofu, Terragrunt joue en partie le rôle des **Stacks** HashiCorp.

PLATEFORMES

TACOS : les alternatives à TFE / HCP Terraform

TACOS = Terraform Automation and Collaboration Software

OUTIL	MODÈLE	HÉBERGEMENT	MOTEURS	POLICY	POINTS FORTS
Atlantis	Open source (Apache 2.0)	Self-hosted	Terraform, OpenTofu	Conftest	Workflow piloté par commentaires de MR/PR (`atlantis plan` / `apply`), simple
Terrakube	Open source (Apache 2.0)	Self-hosted (Kubernetes)	Terraform, OpenTofu	OPA	API compatible TFE, organisations, workspaces, registry privée
Digger	Open source	Dans votre CI	Terraform, OpenTofu	OPA	Réutilise les runners CI existants, locking par projet
Spacelift	Commercial	SaaS ou self-hosted	Terraform, OpenTofu, Pulumi, CFN, K8s, Ansible	OPA	Stacks, dépendances, drift, contributeur majeur d'OpenTofu
env0	Commercial	SaaS (agents self-hosted)	Terraform, OpenTofu, Terragrunt, Pulumi	OPA	FinOps, environnements éphémères, gouvernance des coûts
Scalr	Commercial (free tier)	SaaS (agents self-hosted)	Terraform, OpenTofu	OPA	Modèle proche de TFC, hiérarchie account/environment
Terragrunt Scale	Commercial (free tier)	Dans votre CI	Terraform, OpenTofu	—	GitOps, drift, mises à jour de modules (Gruntwork)
GitLab CI + state GitLab	Inclus GitLab	Self-hosted ou SaaS	Terraform, OpenTofu	Conftest	Backend HTTP, composant CI/CD OpenTofu officiel, environnements
HCP Terraform / TFE	Commercial	SaaS / self-hosted	**Terraform seulement**	Sentinel, OPA	Intégration native, Stacks, private registry

GitLab : les anciens templates Terraform de GitLab ont été retirés au profit d'un **composant CI/CD OpenTofu** (`gitlab.com/components/opentofu`). Le state managé par GitLab fonctionne avec les deux moteurs via le backend `http` .

OUTILLAGE COMPLÉMENTAIRE

Versions, import, éditeur, visualisation, secrets

Gestion des versions

- **tenv** : Terraform, OpenTofu, Terragrunt, Atmos
- **tfenv**, **tfswitch** : Terraform
- Fichiers `.terraform-version` / `.opentofu-version`
- Toujours épingler `required_version`

Import de l'existant

- Blocs `import` + `plan -generate-config-out=gen.tf` (TF et OT)
- `terraform query` (TF 1.14+)
- **Terraformer** : génération massive, maintenance limitée

Éditeur & IA

- **terraform-ls** + extension VS Code HashiCorp
- Extension VS Code et LSP OpenTofu
- **Terraform MCP server**, **OpenTofu Registry MCP server** : docs à jour pour les assistants IA

Visualisation

- `terraform graph -format=mermaid` (1.16)
- `terraform graph | dot -Tsvg`
- **Rover**, **Inframap** : graphes lisibles depuis plan/state

Secrets

- Provider **Vault** / **OpenBao**
- Provider **sops** (fichiers chiffrés dans Git)
- Ephemeral resources + write-only (TF 1.10/1.11, OT 1.11)
- Chiffrement du state (OT)

Drift

- `plan -detailed-exitcode` planifié (code 2 = drift)
- Health assessments (TFE/HCP), drift TACOS
- **driftctl** : non maintenu, à éviter

COMPATIBILITÉ

Matrice outils × Terraform CE / TFE / OpenTofu

OUTIL	TERRAFORM CE	TFE / HCP TERRAFORM	OPENTOFU
Checkov	✓ CLI / CI	✓ Run task Prisma Cloud, ou en CI avant le push	✓ HCL et plan JSON
KICS	✓	✓ En CI (run task via Checkmarx One)	✓
Trivy	✓	✓ En CI ; run task Aqua	✓
TFLint	✓	✓ En CI / pre-commit	✓ (syntaxes exclusives à vérifier)
OPA / Conftest	✓ sur plan JSON	✓ Policy sets OPA natifs	✓ sur plan JSON
Sentinel	✗	✓ Natif	✗
Infracost	✓	✓ Run task	✓
Terragrunt	✓	**Partiel** (runs CLI-driven)	✓ Moteur préféré
Atlantis / Terrakube / Spacelift…	✓	— (alternatives à TFE)	✓
Terratest	✓	✓	✓ (`TerraformBinary: "tofu"`)

Pipeline GitLab CI — socle OpenTofu + state GitLab, qualité

```yaml
# .gitlab-ci.yml — prérequis : backend "http" {} dans infra/
variables:
  TF_ROOT: ${CI_PROJECT_DIR}/infra
  TF_STATE_NAME: production
  TF_ADDRESS: >-
    ${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/terraform/state/${TF_STATE_NAME}

stages: [validate, security, plan, policy, deploy]

.tofu:
  image:
    name: ghcr.io/opentofu/opentofu:1.12.6
    entrypoint: [""]
  before_script:
    - cd "${TF_ROOT}"
    - >-
      tofu init -input=false
      -backend-config="address=${TF_ADDRESS}"
      -backend-config="lock_address=${TF_ADDRESS}/lock"
      -backend-config="unlock_address=${TF_ADDRESS}/lock"
      -backend-config="username=gitlab-ci-token"
      -backend-config="password=${CI_JOB_TOKEN}"
      -backend-config="lock_method=POST"
      -backend-config="unlock_method=DELETE"
      -backend-config="retry_wait_min=5"
```

```yaml
fmt-validate:
  extends: .tofu
  stage: validate
  script:
    - tofu fmt -check -recursive
    - tofu validate

tflint:
  stage: validate
  image:
    name: ghcr.io/terraform-linters/tflint:v0.60.0
    entrypoint: [""]
  script:
    - cd "${TF_ROOT}"
    - tflint --init
    - tflint --recursive --format compact
```

- **State GitLab** : backend `http`, verrouillage géré par GitLab, authentification par `CI_JOB_TOKEN`
- Ancre `.tofu` réutilisée par tous les jobs qui ont besoin du backend
- `entrypoint: [""]` : les images ont la CLI en entrypoint, GitLab a besoin d'un shell

Pipeline GitLab CI — sécurité, plan et policy

```yaml
checkov:
  stage: security
  image:
    # épingler par digest en production
    name: bridgecrew/checkov:latest
    entrypoint: [""]
  script:
    - >-
      checkov -d "${TF_ROOT}"
      --framework terraform
      -o cli -o junitxml
      --output-file-path
      console,checkov.xml
  artifacts:
    when: always
    reports:
      junit: checkov.xml
```

Checkov : rapport JUnit affiché dans l'onglet *Tests* de la MR ; job en échec si une règle échoue.

```yaml
kics:
  stage: security
  image:
    # épingler par digest en production
    name: checkmarx/kics:latest
    entrypoint: [""]
  script:
    - >-
      /app/bin/kics scan
      -p "${TF_ROOT}"
      -o kics-results
      --report-formats "json,sarif"
      --fail-on high
  artifacts:
    when: always
    paths: [kics-results/]
```

KICS : bloquant seulement à partir de HIGH (`--fail-on high`) ; rapports SARIF/JSON en artefacts.

```yaml
plan:
  extends: .tofu
  stage: plan
  script:
    - tofu plan -input=false -out=plan.cache
    - tofu show -json plan.cache > plan.json
  artifacts:
    paths:
      - infra/plan.cache
      - infra/plan.json
    expire_in: 1 day

conftest:
  stage: policy
  image:
    name: openpolicyagent/conftest:latest
    entrypoint: [""]
  needs: [plan]
  script:
    - cd "${TF_ROOT}"
    - conftest test plan.json -p policy/
```

Plan : le fichier `plan.cache` est celui qui sera appliqué ; **Conftest** évalue les règles d'entreprise sur `plan.json`.

Pipeline GitLab CI — coût et apply

```
infracost:
  stage: policy
  image:
    name: infracost/infracost:ci-0.10
    entrypoint: [""]
  needs: [plan]
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  script:
    - cd "${TF_ROOT}"
    - >-
      infracost breakdown --path plan.json
      --format json --out-file infracost.json
    - >-
      infracost comment gitlab --path infracost.json
      --repo "$CI_PROJECT_PATH"
      --merge-request "$CI_MERGE_REQUEST_IID"
      --gitlab-server-url "$CI_SERVER_URL"
      --gitlab-token "$GITLAB_TOKEN" --behavior update

apply:
  extends: .tofu
  stage: deploy
  needs: [plan, conftest, checkov, kics]
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
  environment: production
  script:
    - tofu apply -input=false plan.cache
```

Ce qu'on montre

- Une MR avec un bucket public → **Checkov / KICS rouges**, apply impossible
- Un bucket sans tag `owner` → **Conftest** bloque sur le plan
- Changement de taille d'instance → **commentaire Infracost** sur la MR
- Sur `main` : apply **manuel** du plan exact qui a été validé
- State visible dans *Operate* → *Terraform states*

Points d'attention

- `plan.cache` et `plan.json` peuvent contenir des **secrets** : artefacts courts, projet privé, ou chiffrement du plan (OpenTofu)
- Variables protégées et masquées : `GITLAB_TOKEN`, `INFRACOST_API_KEY` ; credentials cloud via OIDC (`id_tokens`)
- Pour Terraform : image `hashicorp/terraform:1.16` et `terraform` à la place de `tofu`

RECUL

Ce que l'outillage ne fait pas

Limites des contrôles statiques

- Un scanner voit le **code**, pas l'état réel du cloud : une modification manuelle dans la console lui échappe → contrôle du drift nécessaire
- Faux positifs : sans processus d'exception (skip justifié, revue), les équipes finissent par désactiver l'outil
- Les valeurs calculées à l'apply (`known after apply`) ne sont pas vérifiables au plan
- Un module externe non téléchargé n'est pas analysé en mode HCL → scanner le plan

Organisation recommandée

- **Poste :** pre-commit (fmt, TFLint, Checkov) — retour immédiat
- **MR :** scanners + plan + policy + coût — **bloquant** sur HIGH/CRITICAL
- **Main :** apply manuel du plan validé
- **Run :** plan planifié pour détecter le drift ; CSPM cloud en complément
- Un **propriétaire** pour les règles et le registre des exceptions

Défense en profondeur : scanner IaC (prévention) + policy as code (gouvernance) + contrôles du compte cloud (AWS Config, Azure Policy, SCP) qui s'appliquent même si quelqu'un contourne le pipeline.

SYNTHÈSE

À retenir

Terraform CE

- Référence du marché, écosystème le plus large
- BSL 1.1 : sans impact pour un usage interne
- Innovations orientées plateforme HashiCorp (Stacks, actions, query)
- Feuille de route décidée par IBM/HashiCorp

Terraform Enterprise

- Plateforme de gouvernance complète, self-hosted, air-gap
- Sentinel/OPA, SSO/SCIM, audit, private registry, Stacks
- Coût de licence + coût d'exploitation
- Exécute Terraform uniquement

OpenTofu

- Open source MPL 2.0, gouvernance neutre (CNCF)
- Exclusivités utiles : chiffrement du state, provider `for_each`, OCI, `enabled`
- Gouvernance à construire : CI + TACOS
- Migration simple depuis Terraform ≤ 1.9

Écosystème : quasiment tous les outils (Checkov, KICS, Trivy, TFLint, Conftest, Infracost, Terragrunt, TACOS) fonctionnent avec les deux moteurs. Le vrai verrou est la **plateforme** et les **exclusivités** du langage.

Chaîne minimale recommandée : fmt/validate → TFLint → Checkov ou KICS → plan → Conftest → Infracost → apply manuel → drift planifié. Outils épinglés par digest/SHA.

RÉFÉRENCES

Sources (consultées le 5 novembre 2026)

- OpenTofu — blog et archives des versions : opentofu.org/blog
- OpenTofu — releases et CHANGELOG (v1.12, v1.13) : github.com/opentofu/opentofu
- OpenTofu 1.10 : opentofu.org/blog/opentofu-1-10-0
- Terraform — releases 1.14, 1.15, 1.16 : github.com/hashicorp/terraform/releases
- Terraform v1.16.0 released : discuss.hashicorp.com
- Upgrading to Terraform v1.15 : developer.hashicorp.com/terraform/language/upgrade-guides
- Terraform Enterprise releases (1.x, 2.0.x) : developer.hashicorp.com/terraform/enterprise/releases
- Terraform Enterprise 2.0 — HashiCorp Blog (mai 2026)
- HCP Terraform enhanced free tier : hashicorp.com/blog/continuing-hcp-terraform-s-enhanced-free-tier-experience
- HCP Terraform pricing 2026 : scalr.com/learning-center, env0.com/blog/terraform-cloud
- Scanners IaC 2026 (Checkov, KICS, Trivy, Terrascan archivé) : appsecsanta.com/iac-security-tools
- Incident Trivy de mars 2026 et alternatives : codenote.net
- Documentation Checkov : checkov.io
- Documentation KICS : docs.kics.io
- Documentation Trivy : trivy.dev
- Infracost : infracost.io/docs · Terragrunt : terragrunt.gruntwork.io · Conftest : conftest.dev

> Versions et prix évoluent vite : vérifier les pages de release et de tarification avant chaque session de formation.